| Code | Description and Purpose |
| --- | --- |
| import pandas as pd<br><br>import numpy as np<br><br>import matplotlib.pyplot as plt<br><br>import seaborn as sns | These lines of code import the relevant packages to analyze and transform data. |
| sns.set_style('whitegrid')<br><br>sns.set_palette('deep') | These lines of code give the whole document a uniform color palette and style. |
| nhts_data['make'].value_counts() | This counts how many items occur for each listing in the category “make.” |
| print(nhts_data.isnull().sum) | This prints out data with missing values, to be discarded. |
| plt.figure(figsize = (12,6)) | This establishes what the final size of a graph will be. |
| sns.countplot(data = nhts_data, y = 'vehicle_type', order = nhts_data['vehicle_type'].value_counts().index) | This line creates a countplot, or horizontal bar chart, counting the number of each type of vehicle. |
| plt.xlabel('Count', fontsize = 12)<br><br>plt.ylabel('Vehicle Type', fontsize = 12)<br><br>plt.title('Number of Vehicles by Type in NHTS Data', fontsize = 14) | These lines give the figure axis labels and a title. |
| plt.tight_layout()<br><br>plt.show() | These lines establish a tight layout and cause the figure to show onscreen. |
| plt.figure(figsize = (12,6))<br><br>boxplot = nhts_data.boxplot(column = 'vehicle_age', by = 'make') | This creates a boxplot of vehicle ages grouped by their make and sets a size for the figure. |
| plt.xticks(rotation = 90) | This rotates the graph on its side. |

| | |
|---|---|
| plt.xlabel('Make', fontsize = 12)<br><br>plt.ylabel('Vehicle age (years)', fontsize = 12)<br><br>plt.title('Make vs. Vehicle Age (years)', fontsize = 16) | These lines establish axis labels and a title. |
| plt.show() | This prints the graph. |
| plt.figure(figsize = (10,6)) | This sets a final size for the graph. |
| sns.violinplot(data = nhts_data, x = 'household_location', y = 'vehicle_age',<br>hue = 'household_location',<br>inner = 'box', palette = 'Set2', legend = False) | This line creates a violin plot comparing household location with vehicle age, and sets a color scheme. |
| plt.xlabel('Household Location')<br><br>plt.ylabel('Vehicle Age (years)')<br><br>plt.title('Vehicle Age Distribution: Urban vs Rural Households') | These lines give the figure axis labels and a title. |
| plt.show() | This prints the graph. |
| plt.figure(figsize = (10,6))<br><br>plt.hist(nhts_data['vehicle_age'], bins = 20, edgecolor = 'black', alpha = 0.7, color = 'steelblue')<br><br>plt.show() | This creates a histogram showing amounts of vehicle ages, sets a color scheme for it, establishes a final size, and prints the graph. |
| urban_ages = nhts_data[nhts_data['household_location'] == 'Urban']['vehicle_age']<br><br>plt.hist(urban_ages, bins = 20, edgecolor = 'black', alpha = 0.5, color = 'steelblue', label = 'Urban') | These lines first establish a category that is all vehicle ages in urban locations, and then create a histogram showing them and establish a color scheme for it. |
| rural_ages = nhts_data[nhts_data['household_location'] | This creates a category of all vehicle ages in rural locations. |

| | |
|---|---|
| == 'Rural']['vehicle_age'] | |
| plt.hist(rural_ages, bins = 20, edgecolor = 'black', alpha = 0.5, color = 'coral', label = 'Rural')<br><br>plt.legend()<br><br>plt.show() | This creates an overlaid histogram of said rural vehicle ages, gives it a new color scheme and a legend, and prints both graphs. |
| fuel_counts = nhts_data['fuel_type'].value_counts().dropna()<br><br>print(fuel_counts) | These lines count the different amounts of used fuel types, drop missing values, and print. |
| plt.figure(figsize = (10,6))<br><br>fuel_counts.plot(kind = 'bar', color = sns.color_palette('viridis', len(fuel_counts)), edgecolor = 'black') | This creates a bar chart of different amounts of fuel types and gives it a color scheme and size. |
| plt.xlabel('Fuel Type', fontsize = 12)<br><br>plt.ylabel('Number of vehicles', fontsize = 12)<br><br>plt.title('Vehicle Count by fuel type', fontsize = 14)<br><br>plt.xticks(rotation = 45, ha = 'right') | These lines give the chart axis labels, a title, and rotate it 45 degrees. |
| plt.tight_layout()<br><br>plt.show() | These lines restrict the graph’s layout and print it out. |
| nhts_data = nhts_data.dropna(subset = ['make']) | This filters out all of the missing make values. |
| data_grouped = nhts_data.groupby(['make', 'census_division']).size().unstack(fill_value=0) | This groups all the data by both make and census division. |
| plt.figure(figsize = (20,10))<br><br>data_grouped.plot(kind = 'bar', stacked = | This creates a bar chart showing this grouped data and gives it a color scheme and size. |

| | |
|---|---|
| True, color=sns.color_palette('viridis', len(data_grouped.columns))) | |
| plt.xlabel('Make', fontsize =13)<br><br>plt.ylabel('Count', fontsize = 13)<br><br>plt.title('Stacked Bar Chart of Make by Census Division', fontsize = 15)<br><br>plt.xticks(rotation=90) | This gives the graph axis labels and a title, and rotates it onto its side. |
| plt.legend(title = 'Census Division', bbox_to_anchor =(1,1), loc = 'upper left')<br><br>plt.show() | This gives the graph a legend and prints it. |
| type_counts = nhts_data['vehicle_type'].value_counts()<br><br>plt.figure(figsize = (10,8)) | This counts the amount of entries for each vehicle type and establishes the size of a figure to be. |
| plt.pie(type_counts, labels = type_counts.index, autopct = '%1.1f%%', startangle=90,<br>colors=sns.color_palette('Set2', len(type_counts)), textprops={'fontsize':10}) | This creates a pie chart of those amounts of entries and gives it a color scheme. |
| plt.axis('equal')<br><br>plt.title('Distribution of Vehicle Types in NHTS', fontsize = 14)<br><br>plt.tight_layout()<br><br>plt.show() | This gives the chart an axis label, a title, restricts its layout, and prints it. |
| numeric_data = nhts_data.select_dtypes(include = [np.number])<br><br>corr_matrix = numeric_data.corr() | This selects all data columns with numeric values and calculates their correlations to each other. |
| plt.figure(figsize = (12,10))<br><br>sns.heatmap(corr_matrix, annot = True, | This creates a heat map for these correlations of a specific size and layout, and prints it. |

| | |
|---|---|
| cmap = 'coolwarm', fmt = '.2f', linewidth = 0.5, vmin = -1, vmax = 1)<br><br>plt.tight_layout()<br><br>plt.show() | |
| plt.figure(figsize = (12,5))<br><br>sns.lineplot(x = ngsim_data['Time'], y = ngsim_data['follower_acc(m/s^2)']) | This establishes a lineplot of following car acceleration by time and gives the plot a size. |
| plt.xlabel('Time (seconds)', fontsize = 12)<br><br>plt.ylabel('Acceleration (m/s^2)', fontsize = 12)<br><br>plt.title('Follower Acceleration vs. Time', fontsize = 14)<br><br>plt.show() | This gives the chart axis labels, a title, and prints it. |
| plt.figure(figsize = (12,5))<br><br>sns.lineplot(x = ngsim_data['Time'], y = ngsim_data['follower_speed(m/s)']) | This establishes a lineplot of following car speed by time and gives the plot a size. |
| plt.xlabel('Time (seconds)', fontsize = 12)<br><br>plt.ylabel('Speed (m/s)', fontsize = 12)<br><br>plt.title('Follower Speed vs. Time', fontsize = 14)<br><br>plt.show() | This gives the chart axis labels, a title, and prints it. |
| plt.figure(figsize = (12,5))<br><br>sns.lineplot(x = ngsim_data['Time'], y = ngsim_data['follower_position(m)']) | This establishes a lineplot of following car position by time and gives the plot a size. |
| plt.xlabel('Time (seconds)', fontsize = 12)<br><br>plt.ylabel('Position (m)', fontsize = 12)<br><br>plt.title('Follower Position vs. Time', fontsize = 14) | This gives the chart axis labels, a title, and prints it. |

| | |
|---|---|
| plt.show() | |
| trajectory_number = 1 # vehicle pair x<br><br>data_subset = ngsim_data[ngsim_data['trajectory_number'] == trajectory_number] | This restricts drawn values to those with a particular trajectory number. |
| sns.lineplot(x = data_subset['Time'], y = data_subset['leader_position(m)'], label = 'Leader Trajectory')<br><br>sns.lineplot(x = data_subset['Time'], y = data_subset['follower_position(m)'], label = 'Follower Trajectory') | These lines create two lineplots of differing car trajectories, each weighed against time. |
| plt.xlabel('Time (seconds)', fontsize = 12)<br><br>plt.ylabel('Position (m)', fontsize = 12)<br><br>plt.title('Car Following: Trajectory Comparison', fontsize = 14)<br><br>plt.legend(fontsize = 11)<br><br>plt.show() | These give the lineplots axis labels, a title, a legend, and print them. |
| sns.lineplot(x = data_subset['Time'], y = data_subset['leader_speed(m/s)'], label = 'Leader Speed')<br><br>sns.lineplot(x = data_subset['Time'], y = data_subset['follower_speed(m/s)'], label = 'Follower Speed') | These lines create two lineplots of differing car speeds, each weighed against time. |
| plt.xlabel('Time (seconds)', fontsize = 12)<br><br>plt.ylabel('Position (m)', fontsize = 12)<br><br>plt.title('Car Following: Trajectory Comparison', fontsize = 14)<br><br>plt.legend(fontsize = 11)<br><br>plt.show() | These give the lineplots axis labels, a title, a legend, and print them. |

| | |
|---|---|
| sns.lineplot(x = data_subset['Time'], y = data_subset['leader_acc(m/s^2)'], label = 'Leader Acceleration')<br><br>sns.lineplot(x = data_subset['Time'], y = data_subset['follower_acc(m/s^2)'], label = 'Follower Acceleration') | These lines create two lineplots of differing car accelerations, each weighed against time. |
| plt.xlabel('Time (seconds)', fontsize = 12)<br><br>plt.ylabel('Position (m)', fontsize = 12)<br><br>plt.title('Car Following: Trajectory Acceleration Comparison', fontsize = 14)<br><br>plt.legend(fontsize = 11)<br><br>plt.show() | These give the lineplots axis labels, a title, a legend, and print them. |
| gap_distance = data_subset['leader_position(m)'].values - data_subset['follower_position(m)'].values | This calculates the distance between the two cars at all points on the lineplot. |
| plt.figure(figsize = (12,5))<br><br>plt.plot(data_subset['Time'], gap_distance, color = 'purple', linewidth = 1.5) | This establishes a lineplot of the distance between said cars over time, and gives it a color and a size. |
| plt.xlabel('Time (seconds)', fontsize = 12)<br><br>plt.ylabel('Gap Distance (m)', fontsize = 12)<br><br>plt.title('Gap Distance vs. Time', fontsize = 14)<br><br>plt.grid(True, alpha = 0.3)<br><br>plt.show() | This gives the lineplot axis labels, a title, a grid, and prints it. |
| trajectory_number = 1<br><br>data_subset = ngsim_data[ngsim_data['trajectory_number'] == trajectory_number] | This restricts drawn data to that of a specific trajectory number, and then calculates the gap between the positions of the cars following said trajectory. |

| | |
|---|---|
| gap = data_subset['leader_position(m)'].values - data_subset['follower_position(m)'].values<br><br>time = data_subset['Time'].values | |
| fig, axes = plt.subplots(2,2, figsize = (14,10)) | This establishes that there will be multiple plots sharing the same space. |
| axes[0,0].plot(time, data_subset['leader_position(m)'].values, 'b-', label = 'Leader', linewidth = 1.5)<br><br>axes[0,0].plot(time, data_subset['follower_position(m)'].values, 'r-', label = 'Follower', linewidth = 1.5)<br><br>axes[0,0].set_ylabel('Position (m)', fontsize = 11)<br><br>axes[0,0].set_title('Position vs. Time', fontsize = 12)<br><br>axes[0,0].legend(fontsize = 10)<br><br>axes[0,0].grid(True, alpha = 0.3) | These lines create overlapping lineplots of leading and following car positions, and give each the usual trappings. |
| axes[0,1].plot(time, data_subset['leader_speed(m/s)'].values, 'b-', label = 'Leader', linewidth = 1.5)<br><br>axes[0,1].plot(time, data_subset['follower_speed(m/s)'].values, 'r-', label = 'Follower', linewidth = 1.5)<br><br>axes[0,1].set_ylabel('Speed (m/s)', fontsize = 11)<br><br>axes[0,1].set_title('Speed vs. Time', fontsize = 12)<br><br>axes[0,1].legend(fontsize = 10)<br><br>axes[0,1].grid(True, alpha = 0.3) | These lines create overlapping lineplots of leading and following car speeds, and give each the usual trappings. |
| axes[1,0].plot(time, | These lines create overlapping lineplots |

| | |
|---|---|
| data_subset['leader_acc(m/s^2)'].values, 'b-', label = 'Leader', linewidth = 1.5)<br><br>axes[1,0].plot(time, data_subset['follower_acc(m/s^2)'].values, 'r-', label = 'Follower', linewidth = 1.5)<br><br>axes[1,0].set_ylabel('Acceleration (m/s^2)', fontsize = 11)<br><br>axes[1,0].set_title('Acceleration vs. Time', fontsize = 12)<br><br>axes[1,0].legend(fontsize = 10)<br><br>axes[1,0].grid(True, alpha = 0.3) | of leading and following car accelerations, and give each the usual trappings. |
| axes[1,1].plot(time, gap, 'purple', linewidth = 1.5)<br><br>axes[1,1].set_xlabel('Time (seconds)', fontsize =11)<br><br>axes[1,1].set_ylabel('Gap Distance (m)', fontsize = 11)<br><br>axes[1,1].set_title('Gap Distance vs. Time', fontsize = 12)<br><br>axes[1,1].grid(True, alpha = 0.3) | This creates a lineplot showing the gap between the cars, and gives said plot the usual graph necessities. |
| plt.suptitle(f' Vehicle Pair Dashboard - Trajectory {trajectory_number}', fontsize = 16, y=1.01)<br><br>plt.tight_layout()<br><br>plt.show() | This gives a supertitle to all four graphs and prints the whole. |
| trajectory_numbers = [1, 4, 7, 13]<br><br>colors = ['steelblue', 'coral', 'seagreen', 'darkorange'] | These lines restrict drawn data to specific trajectories and applies a color palette. |
| plt.figure(figsize = (12,6)) | This gives a specific size to the figure to come. |

| | |
|---|---|
| for traj, color, in zip(trajectory_numbers, colors):<br>  subset = ngsim_data[ngsim_data['trajectory_number'] == traj]<br>  plt.plot(subset['Time'].values, subset['follower_speed(m/s)'].values, color = color, linewidth = 1.2, label = f'Trajectory {traj}', alpha = 0.8) | This plots the times of four trajectories against each other and gives them different colors. |
| plt.xlabel('Time (seconds)', fontsize = 12)<br><br>plt.ylabel('Follower Speed (m/s)', fontsize = 12)<br><br>plt.title('Follower Speed Comparison Across Vehicle Pairs', fontsize = 14)<br><br>plt.legend(fontsize = 14)<br><br>plt.grid(True, alpha = 0.3)<br><br>plt.show() | This applies the usual graph necessities to the above. |
| plt.figure(figsize = (10,5))<br><br>sns.histplot(ngsim_data['follower_acc(m/s^2)'], bins = 30, kde = True, color = 'steelblue') | This creates a histogram of following car acceleration. |
| plt.figure(figsize = (10,5))<br><br>sns.histplot(ngsim_data['leader_acc(m/s^2)'], bins = 30, kde = True, label = 'Leader Acceleration', color = 'steelblue', alpha = 0.6)<br><br>sns.histplot(ngsim_data['follower_acc(m/s^2)'], bins = 30, kde = True, label = 'Follower Acceleration', color = 'coral', alpha = 0.6) | This creates a histogram of leading car acceleration *and* a histogram of following car acceleration. |
| plt.xlabel('Acceleration (m/s^2)', fontsize = 12)<br><br>plt.ylabel('Frequency', fontsize = 12) | This applies the usual graph necessities. |

| | |
|---|---|
| plt.title('Distribution of Acceleration for Leader and Follower vehicles', fontsize = 14)<br><br>plt.show() | |
| plt.figure(figsize = (10,6))<br><br>sns.scatterplot(x = ngsim_data['leader_speed(m/s)'], y = ngsim_data['leader_acc(m/s^2)'], label = 'Leader', alpha = 0.4, color = 'steelblue', s = 15)<br><br>sns.scatterplot(x = ngsim_data['follower_speed(m/s)'], y = ngsim_data['follower_acc(m/s^2)'], label = 'Follower', alpha = 0.4, color = 'coral', s = 15) | This creates a scatterplot of leader and follower speeds and accelerations. |
| plt.xlabel('Speed (m/s)', fontsize = 12)<br><br>plt.ylabel('Acceleration (m/s^2)', fontsize = 12)<br><br>plt.title('Speed vs Acceleration for Leader and Follower Vehicles', fontsize = 14)<br><br>plt.legend(fontsize = 11)<br><br>plt.show() | This labels the axes of the above as well as applying a title and a legend. |
| trajectory_number = 1<br><br>data_subset = ngsim_data[ngsim_data['trajectory_number'] == trajectory_number] | This restricts drawn trajectories. |
| leader_plot = plt.scatter(data_subset['leader_position(m)'], data_subset['Time'], c=data_subset['leader_speed(m/s)'],cmap='plasma',s=20,label='Leader') | This creates a scatter plot of leader speed, leader position, follower speed, and follower position weighed against time. |

| | |
|---|---|
| plt.scatter(data_subset['follower_position(m)'],data_subset['Time'], c=data_subset['follower_speed(m/s)'],cmap='plasma',s=20,label='Follower') | |
| cbar = plt.colorbar(leader_plot)<br><br>cbar.set_label('Speed (m/s)') | This changes the colors of the above. |
| plt.xlabel('Position (m)', fontsize=12)<br><br>plt.ylabel('Time (s)', fontsize=12)<br><br>plt.title('Trajectory of Leader and Follower Vehicles Colored by Speed', fontsize=14)<br><br>plt.legend()<br><br>plt.grid(True, alpha=0.3)<br><br>plt.show() | This gives the plot the usual graphing necessities. |
| plt.figure(figsize = (12,6)) | This gives the figure a size. |
| plt.plot(time_data, data_subset['follower_position(m)'].values,'b-', linewidth = 2, label = 'Real Follower (NGSIM)')<br><br>plt.plot(time_data, leader_position, 'k--', linewidth = 2, label = 'Leader (NGSIM)')<br><br>plt.plot(time_data, sim_position, 'r-', linewidth = 2,  label = 'Simulated Follower') | This creates a plot of leader and follower positions as well as simulated follower positions. |
| plt.xlabel('Time (s)', fontsize = 12)<br><br>plt.ylabel('Position (m)', fontsize = 12)<br><br>plt.legend()<br><br>plt.title('Position vs Time', fontsize = 14) | This gives the graph all the necessary labels. |
| plt.grid(True, alpha = 0.5)<br><br>plt.show() | This prints the graph. |